Constructors and Their Types in C#

In C#, a **constructor** is a special method used to initialize objects when they are created.
Unlike normal methods, a constructor:

- Has the **same name as the class or struct**.
- Does **not have a return type** (not even void).
- Is called **automatically** when an instance is created.

The primary purpose of a constructor is to set default values or perform any setup steps required before the object is used.

Syntax of a Constructor

When you create an object, the constructor runs automatically.

Types of Constructors in C#

a) Default Constructor

- Takes no parameters.
- Initializes fields to their default values (0, null, false, etc.).
- If you do not define any constructor, C# provides a default one automatically.

---

b) Parameterized Constructor

- Accepts parameters to initialize an object with specific values.
- Allows flexibility when creating instances.

---

c) Copy Constructor

- Creates a new object by copying values from another object of the same type.
- Not built-in automatically; you define it yourself.

---

d) Static Constructor

- Used to initialize **static members** of a class.
- Executes only **once** per type, before any instance or static member is accessed.

- Cannot take parameters.

---

e) Private Constructor

- Prevents direct creation of objects from outside the class.
- Commonly used in **Singleton Design Pattern**.

Constructors in Structs

- Structs can have parameterized constructors, but **cannot** define an explicit parameterless constructor.
- C# automatically provides a default constructor that sets all fields to default values.

Why Constructors Are Important

- **Automatic Initialization**: Objects start in a valid state.
- **Flexibility**: Multiple constructors (overloading) allow different initialization options.
- **Readability**: Makes the intent of object creation clear.
- **Encapsulation**: Can hide complex setup logic inside the constructor.